Federated Learning

FL Algorithms

Assoc. Prof. Alexander Jung

21 March 2026 – IMC Krems

Glossary **Book** **GitHub**

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Table of Contents

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Your Opinion Matters

- In the next slides, I will discuss some of the feedback collected via Teams after our first session.
- Each slide contains a quote from the feedback and then I will explain how I plan to address it.

Recordings

"Would appreciate a recording of each session.. . . "

If all of you agree, I will try to record all sessions from now on (using zoom). I could post them on my youtube channel or in Teams.

Course Logistics

> *"... Today, we discussed about how the semester would go and some details about the project but I think we all forgot about it after the class... Would also appreciate a more detailed project expectations and requirements. "*

I have significantly revised the project description and requirements in the course logistics slides. You can find them in the Teams channel.

FL Applications - Why does it Matter?

"Maybe a better section with some practical applications of the concepts mentioned, to more explicitly show to the students: "This matters". There is a short mention of the practical applications, but it does not name the exact use cases/downsides/benefits/alternatives."

"More real-world case studies and discussion of practical challenges like privacy risks."

FL shines when data is not centrally available, like in a big database, but fragmented across local/private datasets with sensitive information (healthcare records). We will dedicate an entire session to privacy protection in FL.

From Math to Python

"... I would like is to have a bit more of an example approach ... correlating the mathematical part to its use in real life. "

"A simple code example, especially one linked to the key steps in the course documentation, could help ..."

"... more mini coding examples (ex:- how the laplacian matrix is computed through a numpy example perhaps) "

"Hands on activities during the session where the lecturer shows a working example in code."

This session will include more coding demos.

FL Coding Frameworks

> *"...comparisons between different FL frameworks (tensorflow vs pytorch based implementations) that helps to understand the connection between theory and real world software development or system architecture "*

This session will include a demo where one device uses decision tree and other model uses linear model. In principle they can use also different Python libraries or even completely different programming frameworks.

Slowing Down

"Felt pretty fast going from one concept to the next and therefore was pretty hard to follow and not just stop listening at some point because you don't even really know anymore what to ask to make the confusion clearer."

"More step by step explanations of the mathematics concepts like the Laplacian matrix and federated learning network properties."

We will slow down the pace and recap important concepts from the first session, also using live coding demos.

Explicit Convergence Conditions

"Explicit convergence conditions with actual numbers, practical examples of heterogeneous models (tree + linear model together)"

I will discuss convergence rates in the context of GD for lin.reg. and then more generaly in terms of fixed-point iterations.

Fixed-Point Iteration in FL

"Add more runnable code for GTVMin/fixed-point iteration in L2, give a concrete algorithm for choosing edge weights instead of just statistical tests."

This session will include a coding demo for how to implement ML as fixed-point iteration using gradient descent (GD) for linear regression.

Table of Contents

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

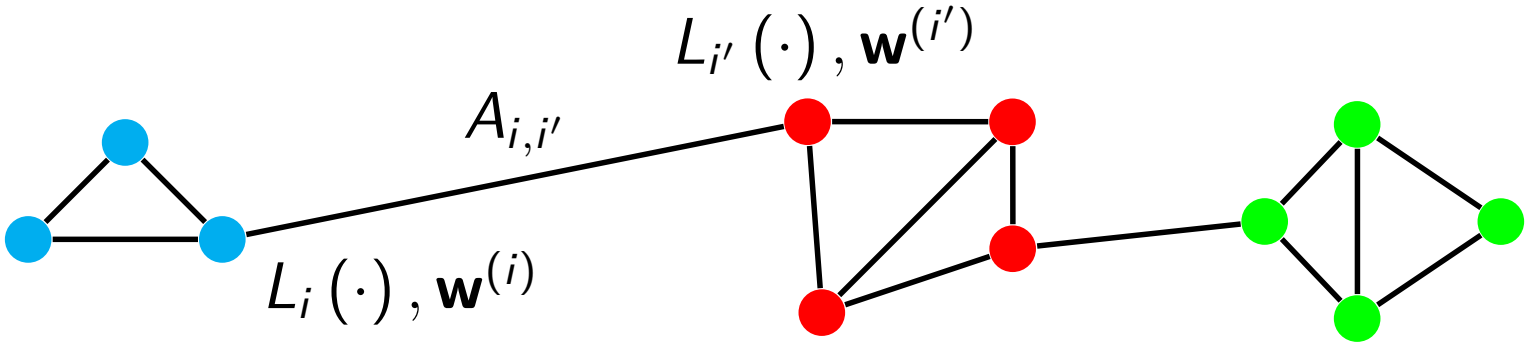
A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Recap: FL Network



- devices $i = 1, \ldots, n$ connected by edges $\{i, i'\} \in \mathcal{E}$
- each device i holds **local data** and trains local model $\mathbf{w}^{(i)}$
- edge weight $A_{i,i'} > 0$ encodes similarity between devices

FL Network in Python

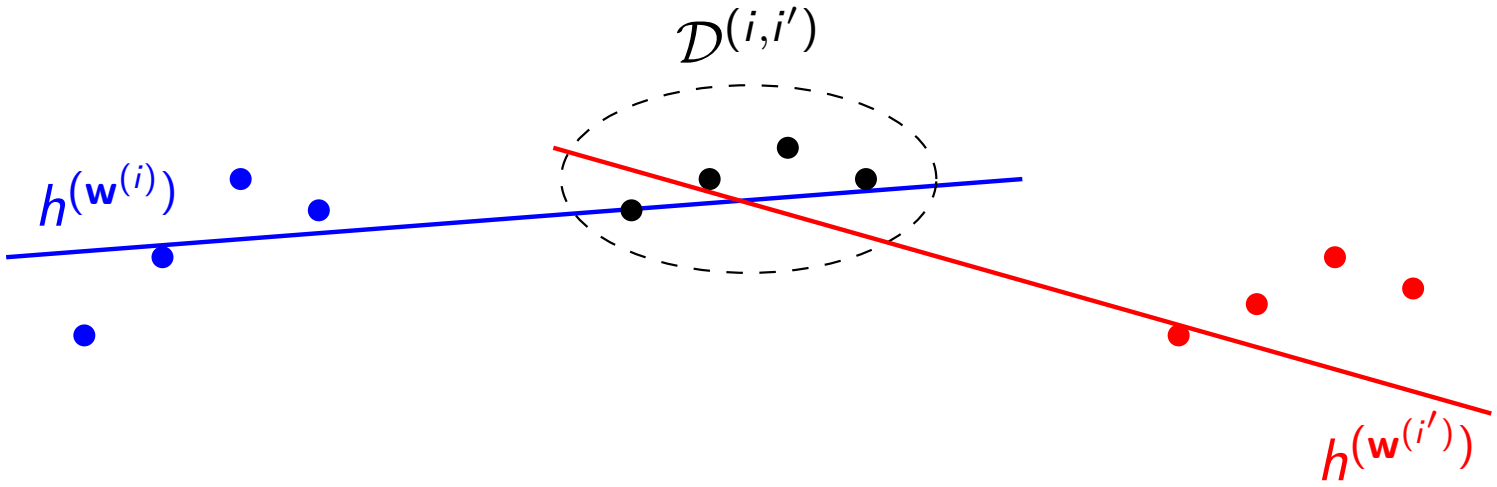
```python
import networkx as nx
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

G = nx.Graph()

# Each node = one device; attach local data, model, and loss
G.add_node(1, data=(X1, y1),
              model=LinearRegression(),
              loss=mean_squared_error)
G.add_node(2, data=(X2, y2),
              model=DecisionTreeRegressor(max_depth=3),
              loss=mean_squared_error)
G.add_node(3, data=(X3, y3),
              model=LinearRegression(),
              loss=mean_squared_error)

# Edges carry similarity weights
G.add_edge(1, 2, weight=0.8)
G.add_edge(2, 3, weight=0.3)
```

Node attributes: `data` (private), `model` (sklearn), `loss` (callable).

Edge attribute: `weight` $= A_{i,i'} > 0$.

Building a Penalty Across Edges

- consider two connected nodes i, i' with datasets $\mathcal{D}^{(i)}, \mathcal{D}^{(i')}$
- assume a non-empty overlap $\mathcal{D}^{(i,i')} = \mathcal{D}^{(i)} \cap \mathcal{D}^{(i')}$



quantify discrepancy between i and i' via

$$\sum_{\mathbf{x} \in \mathcal{D}^{(i,i')}} \left(h^{(\mathbf{w}^{(i)})}(\mathbf{x}) - h^{(\mathbf{w}^{(i')})}(\mathbf{x}) \right)^2 = \sum_{\mathbf{x} \in \mathcal{D}^{(i,i')}} \left(\mathbf{x}^T \mathbf{w}^{(i)} - \mathbf{x}^T \mathbf{w}^{(i')} \right)^2$$

$$= \left(\mathbf{w}^{(i)} - \mathbf{w}^{(i')} \right)^T \left[\sum_{\mathbf{x} \in \mathcal{D}^{(i,i')}} \mathbf{x}\mathbf{x}^T \right] \left(\mathbf{w}^{(i)} - \mathbf{w}^{(i')} \right).$$

Discrepancy Measure for Parametric models

use "norm-like" function $\phi(\mathbf{w}^{(i)} - \mathbf{w}^{(i')})$ of difference between model parameter at two nodes i, i'

- $\phi(\mathbf{u}) = \|\mathbf{u}\|_2^2$, or
- $\phi(\mathbf{u}) = \mathbf{u}^T \mathbf{Q} \mathbf{u}$ with positive semi-definite (psd) $\mathbf{Q}$, or
- $\phi(\mathbf{u}) = \|\mathbf{u}\|_1$, or
- $\phi(\mathbf{u}) = \|\mathbf{u}\|$, or
- ...

Recap: GTVMin Design Principle

Couple local model training across the network:

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}} \underbrace{\sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right)}_{\text{fit local data}} + \alpha \underbrace{\sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'} \left\|\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right\|_2^2}_{\text{penalise disagreement}} \tag{GTVMin}$$

Parameter α controls the trade-off:

- $\alpha = 0$: fully independent local training
- $\alpha \to \infty$: forces all models to agree (global model)

From Optimization to Algorithm – Our Workflow

Stack all local model parameters: $\mathbf{w} = \left(\mathbf{w}^{(1)}, \ldots, \mathbf{w}^{(n)}\right)$.

1. **Formulate** the optimization problem (GTVMin)

$$\min_{\mathbf{w}} \sum_{i} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{\{i,i'\}\in\mathcal{E}} A_{i,i'} \left\|\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right\|_2^2$$

2. **Derive** necessary optimality conditions $\Rightarrow$ fixed-point characterisation $\widehat{\mathbf{w}} = \mathcal{F}(\widehat{\mathbf{w}})$, where the **global operator** $\mathcal{F}$ is assembled node-wise:

$$\mathcal{F}(\mathbf{w}) = \begin{pmatrix} \mathcal{F}^{(1)}(\mathbf{w}) \\ \vdots \\ \mathcal{F}^{(n)}(\mathbf{w}) \end{pmatrix}$$

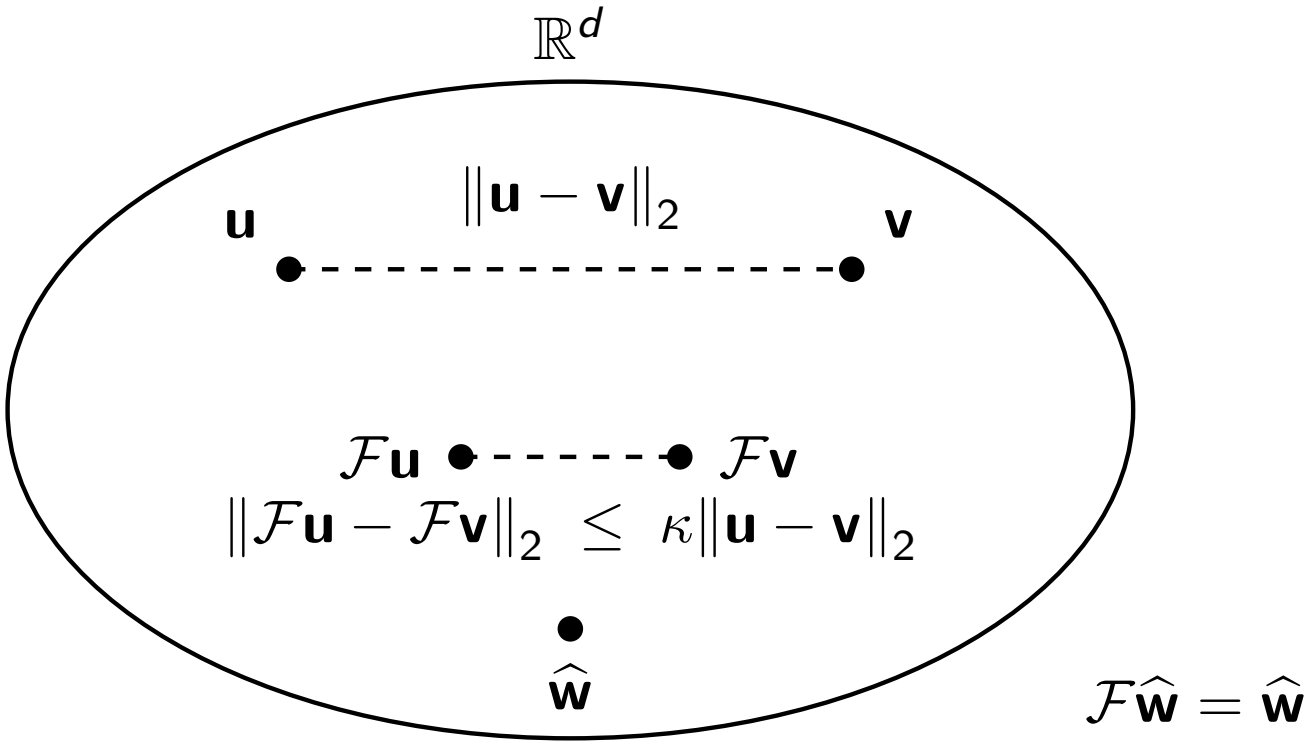
3. **Verify** $\mathcal{F}$ is a contractive operator $\Rightarrow$ unique $\widehat{\mathbf{w}}$, guaranteed convergence.

4. **Iterate** $\mathbf{w}^{(t+1)} = \mathcal{F}\left(\mathbf{w}^{(t)}\right)$

Contractions

Definition. An operator $\mathcal{F} : \mathbb{R}^d \to \mathbb{R}^d$ is a *contractive operator* if $\exists\, \kappa \in [0, 1)$ such that

$$\|\mathcal{F}\,\mathbf{u} - \mathcal{F}\,\mathbf{v}\|_2 \;\leq\; \kappa\,\|\mathbf{u} - \mathbf{v}\|_2 \qquad \forall\, \mathbf{u}, \mathbf{v} \in \mathbb{R}^d.$$



Banach's Fixed-Point Theorem

If $\mathcal{F}$ is a contraction ($\kappa < 1$), then

- there exists a **unique** fixed point $\widehat{\mathbf{w}} = \mathcal{F}\,\widehat{\mathbf{w}}$, and
- iterates $\mathbf{w}^{(t+1)} = \mathcal{F}\,\mathbf{w}^{(t)}$ converge to $\widehat{\mathbf{w}}$
- with **geometric rate**: $\left\|\mathbf{w}^{(t+1)} - \widehat{\mathbf{w}}\right\|_2 \leq \kappa \left\|\mathbf{w}^{(t)} - \widehat{\mathbf{w}}\right\|_2$.

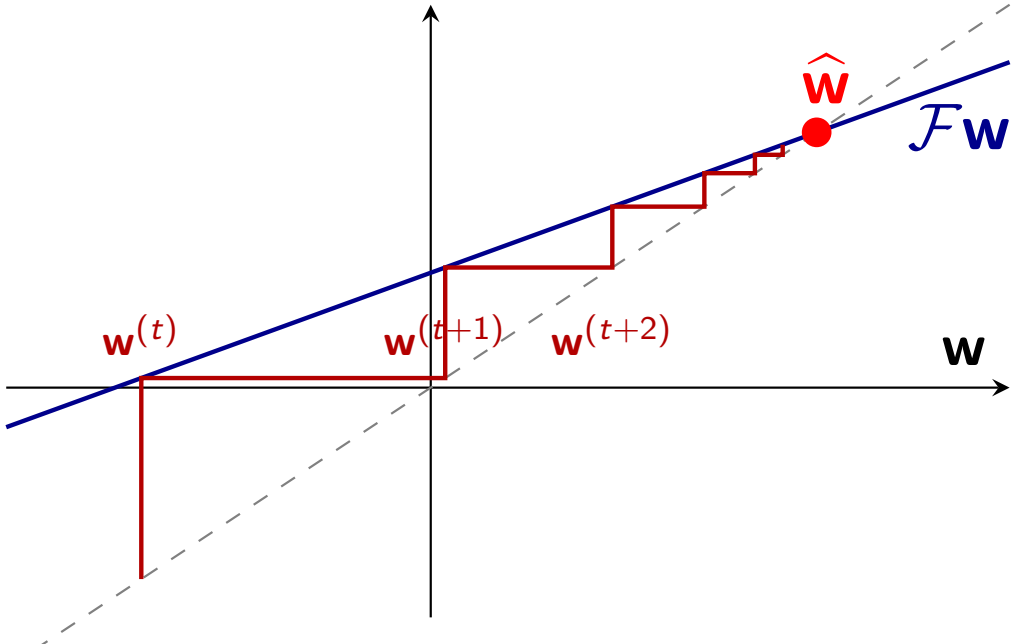


Table of Contents

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Walk-Through: GD for Linear Regression

Let's illustrate the workflow using the example of GD for linear regression.

Example: Single Node, Linear Least Squares

One device, no edges. Local loss: $L_1(\mathbf{w}) = \frac{1}{m}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$.

1. **Formulate.** GTVMin collapses to plain least squares:

$$\min_{\mathbf{w}\in\mathbb{R}^d} \frac{1}{m}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

2. **Fixed-point characterization.** Necessary condition $\nabla L_1(\mathbf{w}) = \mathbf{0} \;\Rightarrow\; \mathbf{X}^T\mathbf{X}\,\widehat{\mathbf{w}} = \mathbf{X}^T\mathbf{y}$ (normal equations). GD step is affine: $\mathcal{F}\,\mathbf{w} = \underbrace{\left(\mathbf{I} - \frac{2\eta}{m}\mathbf{X}^T\mathbf{X}\right)}_{\mathbf{F}}\mathbf{w} + \frac{2\eta}{m}\mathbf{X}^T\mathbf{y}$

3. **Contraction check.** $\mathcal{F}$ is a contraction $\iff \|\mathbf{F}\|_2 < 1$ $\iff \eta < \frac{m}{\lambda_{\max}(\mathbf{X}^T\mathbf{X})}$

4. **Iterate.** $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{2\eta}{m}\mathbf{X}^T\left(\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y}\right)$ $\qquad \leftarrow$ GD

Gradient Step as an Affine Map

Expand (GD) for the linear squared-loss:

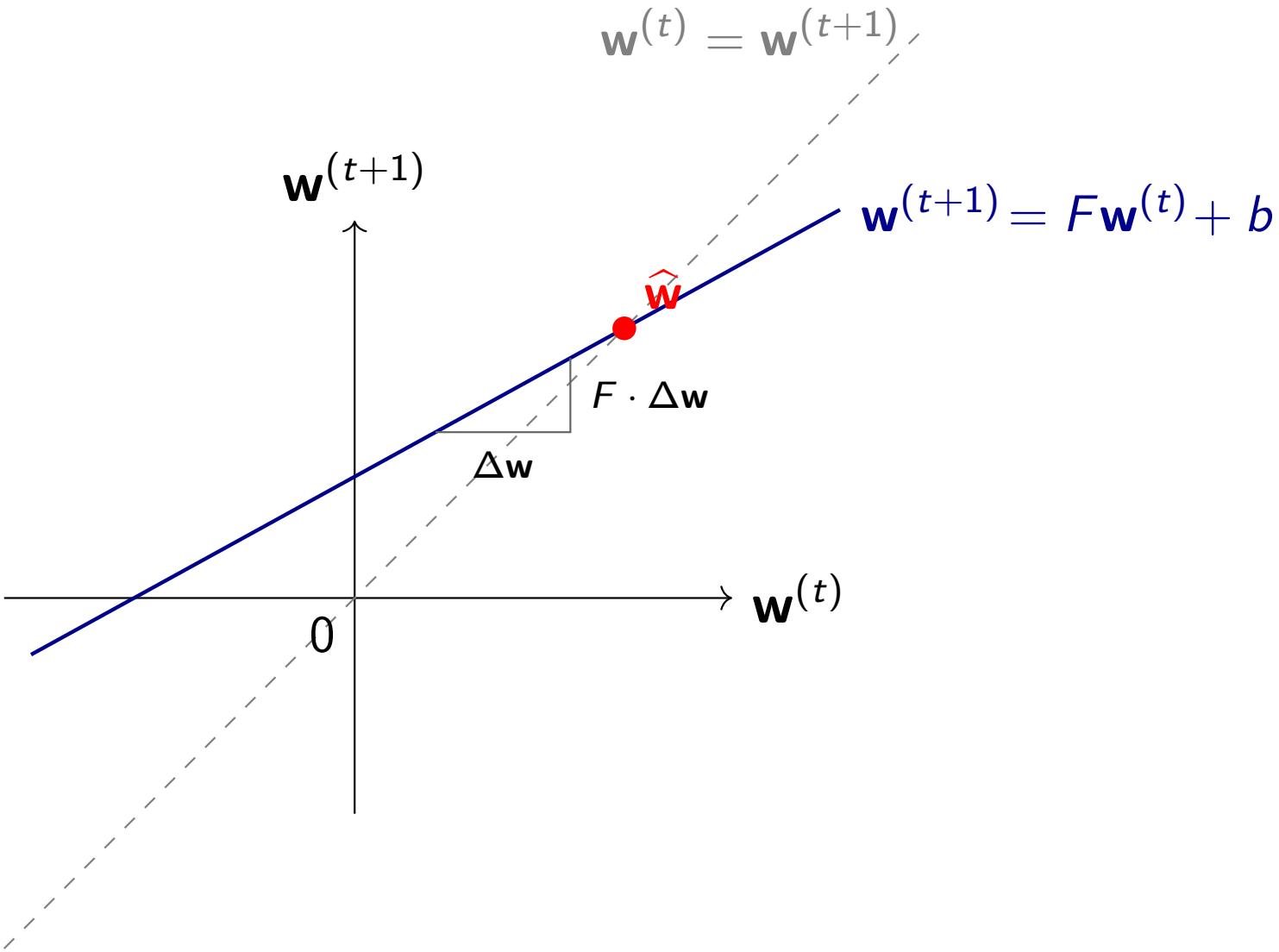
$$\begin{aligned}\mathbf{w}^{(t+1)} &= \mathbf{w}^{(t)} - \frac{2\eta}{m}\mathbf{X}^T\big(\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y}\big) \\ &= \underbrace{\left(\mathbf{I} - \tfrac{2\eta}{m}\mathbf{X}^T\mathbf{X}\right)}_{\mathbf{F}}\mathbf{w}^{(t)} + \underbrace{\tfrac{2\eta}{m}\mathbf{X}^T\mathbf{y}}_{\mathbf{b}}\end{aligned}$$

One gradient step = application of the **affine map** $\mathcal{F}\,\mathbf{w} = \mathbf{F}\mathbf{w} + \mathbf{b}$.

Fixed point $\widehat{\mathbf{w}} = \mathbf{F}\widehat{\mathbf{w}} + \mathbf{b} \iff \mathbf{X}^T\mathbf{X}\,\widehat{\mathbf{w}} = \mathbf{X}^T\mathbf{y}$ (normal equations).

Affine Map: Geometric Picture (scalar case)

For $d = 1$ the update $\mathbf{w}^{(t+1)} = F\,\mathbf{w}^{(t)} + b$ is a straight line with **slope** F and **intercept** b.



Spectral Norm of $\mathbf{F}$ and the Step-Size Condition

The matrix $\mathbf{F} = \mathbf{I} - \frac{2\eta}{m}\mathbf{X}^T\mathbf{X}$ is symmetric. Its eigenvalues are

$$\lambda_k(\mathbf{F}) = 1 - \frac{2\eta}{m}\,\lambda_k(\mathbf{X}^T\mathbf{X}), \quad k = 1, \ldots, d.$$

The spectral norm equals the largest absolute eigenvalue:

$$\|\mathbf{F}\|_2 = \max_k \left|1 - \frac{2\eta}{m}\,\lambda_k(\mathbf{X}^T\mathbf{X})\right|.$$

$\mathcal{F}$ is a **contraction** $\iff \|\mathbf{F}\|_2 < 1 \iff$ all eigenvalues of $\mathbf{F}$ lie in $(-1, 1) \iff$

$$\eta \;<\; \frac{m}{\lambda_{\max}(\mathbf{X}^T\mathbf{X})}.$$

Eigenvalues of $\mathbf{F}$ as a Function of η

Since $\mathbf{F} = \mathbf{I} - \frac{2\eta}{m}\mathbf{X}^T\mathbf{X}$, its eigenvalues are **linear** in η:

$$\lambda_k(\mathbf{F}) = 1 - \frac{2\eta}{m}\,\lambda_k(\mathbf{X}^T\mathbf{X}).$$

- Large $\lambda_k(\mathbf{X}^T\mathbf{X}) \Rightarrow$ eigenvalue decreases faster
- Contraction requires all $\lambda_k(\mathbf{F}) \in (-1, 1)$
- λ_{max} reaches -1 at $\eta_{\text{max}} = m/\lambda_{\text{max}}$ (divergence)
- η^* balances extremes: both hit $\pm\frac{\kappa-1}{\kappa+1}$ simultaneously

Optimal Step Size and Contraction Rate

Assume $\lambda_{\text{min}}(\mathbf{X}^T\mathbf{X}) > 0$ (full column rank). For

$$\eta^* = \frac{m}{\lambda_{\text{max}} + \lambda_{\text{min}}}.$$

the resulting contraction rate is

$$\|\mathbf{F}^*\|_2 = \frac{\lambda_{\text{max}} - \lambda_{\text{min}}}{\lambda_{\text{max}} + \lambda_{\text{min}}} = \frac{\kappa - 1}{\kappa + 1} \qquad (\kappa\text{-rate})$$

where $\kappa = \lambda_{\text{max}}/\lambda_{\text{min}} \geq 1$ is the **condition number** of $\mathbf{X}^T\mathbf{X}$.

Well-conditioned data ($\kappa \approx 1$) $\Rightarrow$ rate ≈ 0 (fast).
Ill-conditioned data ($\kappa \gg 1$) $\Rightarrow$ rate ≈ 1 (slow).

When Is $\kappa \approx 1$? (Well-Conditioned Data)

$\kappa = \lambda_{\max}(\mathbf{X}^T\mathbf{X}) \,/\, \lambda_{\min}(\mathbf{X}^T\mathbf{X}) \approx 1$ when all eigenvalues of $\mathbf{X}^T\mathbf{X}$ are roughly equal. This requires:

- **Similar feature variances.** Each feature column has comparable variance.
- **Uncorrelated features.** Columns of $\mathbf{X}$ nearly orthogonal $\Rightarrow$ $\mathbf{X}^T\mathbf{X} \approx \sigma^2\mathbf{I}$, so $\kappa \approx 1$.
- **How to achieve it.** Standardize each feature (subtract mean, divide by std) before training.

Well-conditioned data $\Rightarrow$ contraction rate $\kappa_{\text{rate}} \approx 0 \Rightarrow$ GD converges in very few steps.

When Is $\kappa \gg 1$? (Ill-Conditioned Data)

$\kappa \gg 1$ when eigenvalues of $\mathbf{X}^T\mathbf{X}$ span many orders of magnitude.

- **Features on very different scales.** E.g. age (years) alongside income (euros): $\lambda_{\text{max}}/\lambda_{\text{min}} \sim 10^6$, rate ≈ 1.
- **Highly correlated features.** One column of $\mathbf{X}$ nearly a linear combination of others $\Rightarrow$ $\lambda_{\text{min}} \approx 0$, $\kappa \to \infty$.

Ill-conditioned data $\Rightarrow$ contraction rate $\kappa_{\text{rate}} \approx 1 \Rightarrow$ GD is slow regardless of how η is chosen.

Table of Contents

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Fixed-Point Characterization of GTVMin

Recall GTVMin:

$$\min_{\mathbf{w}^{(1)},\ldots,\mathbf{w}^{(n)}} \sum_{i=1}^{n} L_i\left(\mathbf{w}^{(i)}\right) + \alpha \sum_{\{i,i'\}\in edges} A_{i,i'} \left\|\mathbf{w}^{(i)} - \mathbf{w}^{(i')}\right\|_2^2 \tag{GTVMin}$$

A necessary condition for a GTVMin solution $\widehat{\mathbf{w}}^{(i)}$:

$$\widehat{\mathbf{w}}^{(i)} = \mathcal{F}^{(i)}\left(\widehat{\mathbf{w}}^{(1)},\ldots,\widehat{\mathbf{w}}^{(n)}\right) \quad \text{for } i = 1,\ldots,n \tag{NecCond}$$

with

$$\mathcal{F}^{(i)}\left(\widehat{\mathbf{w}}^{(1)},\ldots,\widehat{\mathbf{w}}^{(n)}\right) := \operatorname*{arg\,min}_{\mathbf{v}\in\mathbb{R}^d} L_i\left(\mathbf{v}\right) + \alpha \sum_{i'\in\mathcal{N}^{(i)}} A_{i,i'} \left\|\mathbf{v} - \widehat{\mathbf{w}}^{(i')}\right\|_2^2$$

Note: $\mathcal{F}^{(i)}$ depends only on parameters of neighbours $\mathcal{N}^{(i)}$.

Core Step of FL Algorithms

at time t, each device i:

- receives neighbours' parameters $\mathbf{w}^{(i',t)}$ for all $i' \in \mathcal{N}^{(i)}$
- computes the update

$$\mathbf{w}^{(i,t+1)} = \mathcal{F}^{(i)}\big(\mathbf{w}^{(1,t)}, \ldots, \mathbf{w}^{(n,t)}\big) \qquad \text{(FLAlgo)}$$

- shares $\mathbf{w}^{(i,t+1)}$ with neighbours and waits for the next iteration.

Different choices of $\mathcal{F}^{(i)}$ yield different FL algorithms.

Core Step in Python

```
# one iteration on device i

# 1. READ  -- receive current params from neighbours
nb_params = {nb: receive(nb) for nb in neighbours}

# 2. UPDATE  -- apply local operator F^(i) to all params
params = update_op(params, local_data,
                   nb_params, edge_weights)

# 3. SHARE  -- broadcast updated params
for nb in neighbours:
    send(nb, params)
```

Table of Contents

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Recap: FL as Fixed-Point Iterations

Synchronous FL algorithm: all devices update simultaneously,

$$\mathbf{w}^{(i,t+1)} = \mathcal{F}^{(i)}\big(\mathbf{w}^{(1,t)}, \ldots, \mathbf{w}^{(n,t)}\big)$$

Algorithm	**Choice of $\mathcal{F}^{(i)}$**
FedGD	full gradient step
FedSGD	stochastic gradient step
FedAvg	multiple local SGD steps + averaging
FedProx	proximal step
FedRelax	Gauss–Seidel relaxation

All of these algorithms are instances of (FLAlgo) – they differ only in the operator.

Federated Gradient Descent (FedGD)

Each device $i = 1, \ldots, n$ initialises $\mathbf{w}^{(i,0)} := \mathbf{0}$, $t := 0$.

Repeat at each device i:

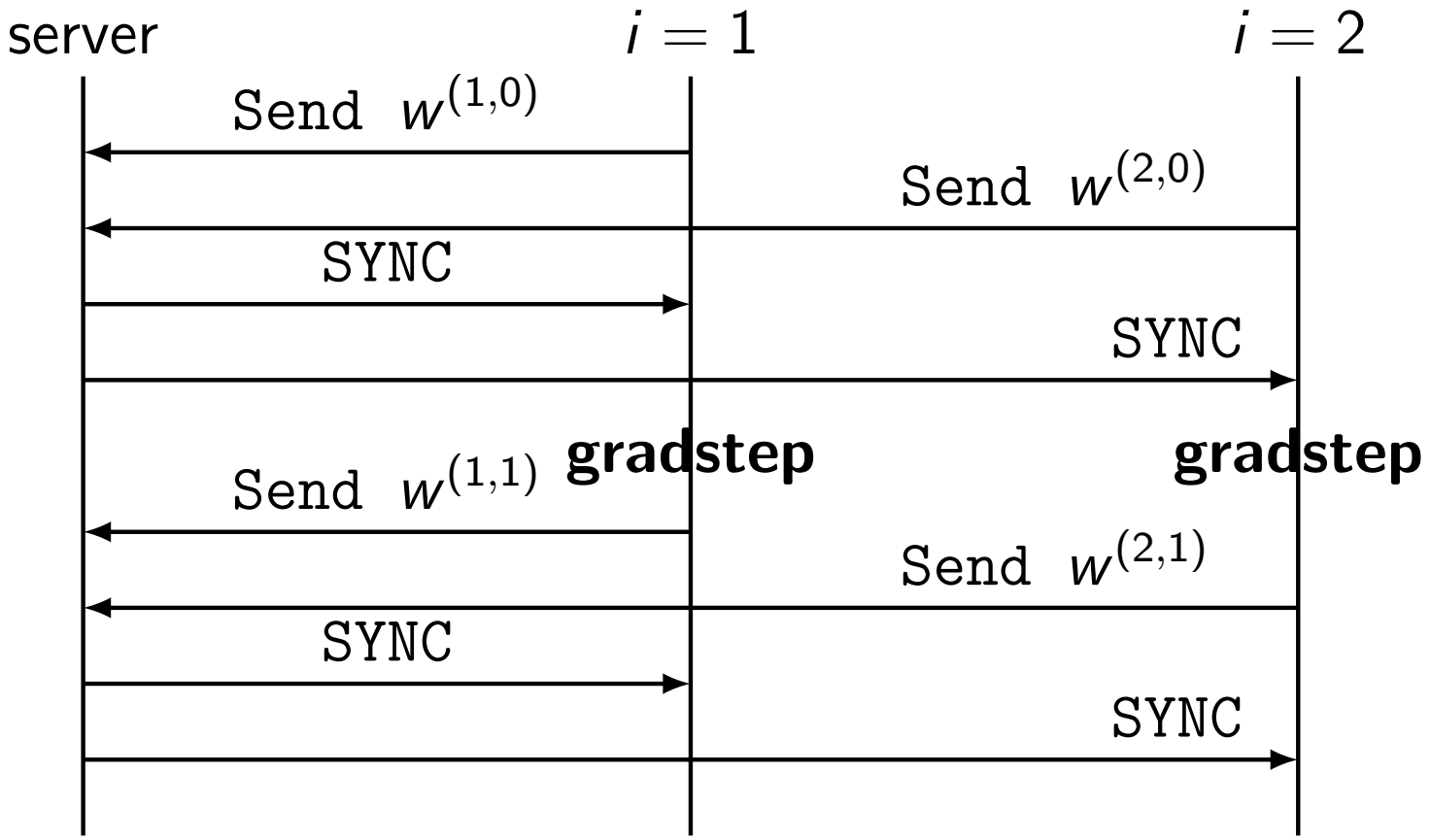
1. **Share.** Send $\mathbf{w}^{(i,t)}$ to all neighbours $i' \in \mathcal{N}^{(i)}$
2. **Gradient step.**

$$\mathbf{w}^{(i,t+1)} = \mathbf{w}^{(i,t)} - \eta_{t,i} \left[\nabla L_i \left(\mathbf{w}^{(i,t)} \right) + 2\alpha \sum_{i' \in \mathcal{N}^{(i)}} A_{i,i'} \left(\mathbf{w}^{(i,t)} - \mathbf{w}^{(i',t)} \right) \right]$$

3. **Clock tick.** $t := t + 1$

Caution: all devices must execute steps synchronously.

Implementing FedGD with a Sync Server



Python demo

Federated SGD (FedSGD)

Problem with FedGD: computing the full gradient

$$\nabla L_i\left(\mathbf{w}^{(i)}\right) = \frac{-2}{m_i}\sum_{r=1}^{m_i}\mathbf{x}^{(i,r)}\left(y^{(i,r)} - \left(\mathbf{w}^{(i)}\right)^T\mathbf{x}^{(i,r)}\right)$$

requires a pass over *all* m_i local data points – expensive or infeasible.

Solution – stochastic approximation: replace the full sum with a sum over a random *batch* $\mathcal{B}^{(i)} \subseteq \{1, \ldots, m_i\}$:

$$\nabla L_i\left(\mathbf{w}^{(i)}\right) \approx \frac{-2}{|\mathcal{B}^{(i)}|}\sum_{r\in\mathcal{B}^{(i)}}\mathbf{x}^{(i,r)}\left(y^{(i,r)} - \left(\mathbf{w}^{(i)}\right)^T\mathbf{x}^{(i,r)}\right)$$

FedAvg

Setting: all devices share a common global model, $\mathbf{w}^{(i)} = \mathbf{w}^{(i')} \; \forall \, i, i'$.

Init counter $t := 0$ and global params $\widehat{\mathbf{w}} := \mathbf{0}$.

1. **Broadcast.** server sends $\widehat{\mathbf{w}}$ to all devices
2. **Local updates.** each device computes *multiple* local SGD steps starting from $\widehat{\mathbf{w}}$
3. **Collect.** devices send updated $\mathbf{w}^{(i,t)}$ to server
4. **Aggregate.** server computes $\widehat{\mathbf{w}} := \frac{1}{n} \sum_{i=1}^{n} \mathbf{w}^{(i,t)}$
5. **Clock tick.** $t := t + 1$; go to 1

Key modifications over FedGD: stochastic gradients, multiple local steps, subset of active devices per round.

B. McMahan et al., *Communication-Efficient Learning of Deep Networks from Decentralized Data*, PMLR 2017.

FedProx

Problem with FedAvg: multiple local steps on heterogeneous devices can cause *client drift* – local models diverge far from the global model.

FedProx fix: replace local SGD steps with a *proximal* update

$$\mathbf{w}^{(i)} := \arg\min_{\mathbf{v}\in\mathbb{R}^d} \left[L_i(\mathbf{v}) + \frac{1}{\eta}\|\mathbf{v} - \widehat{\mathbf{w}}\|_2^2 \right]$$

where $\widehat{\mathbf{w}}$ is the current global model.

The proximal term $\frac{1}{\eta}\|\mathbf{v} - \widehat{\mathbf{w}}\|_2^2$ **anchors** each device's update close to the global model.

Benefit: more robust and less sensitive to tuning when devices are heterogeneous.

T. Li et al., *Federated Optimization in Heterogeneous Networks*, MLSys 2020.

FedRelax

Idea: decouple the coupling terms in GTVMin by treating neighbours' parameters as constants.

Algorithm (Gauss–Seidel / Jacobi relaxation):

1. **Share.** each device i shares $\mathbf{w}^{(i,t)}$ with neighbours
2. **Local update.** each device i solves

$$\mathbf{w}^{(i,t+1)} := \arg\min_{\mathbf{v}\in\mathbb{R}^d} L_i(\mathbf{v}) + \alpha \sum_{i'\in\mathcal{N}^{(i)}} A_{i,i'} \left\| \mathbf{v} - \mathbf{w}^{(i',t)} \right\|_2^2$$

3. **Clock tick.** $t := t + 1$

Note: the local update is structurally identical to *ridge regression* – it can be solved in closed form for linear models.

Table of Contents

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Why Asynchronous FL?

Challenges of synchronous execution:

- devices have heterogeneous computational resources
- evaluating the local loss may require ongoing data collection
- message passing over unreliable wireless links
- devices may join or drop out spontaneously (battery, mobility)

$\Rightarrow$ In practice, **waiting** for the slowest device (straggler) can dominate runtime.

Asynchronous FL lets each device update **whenever it is ready**, using possibly outdated parameters from its neighbours.

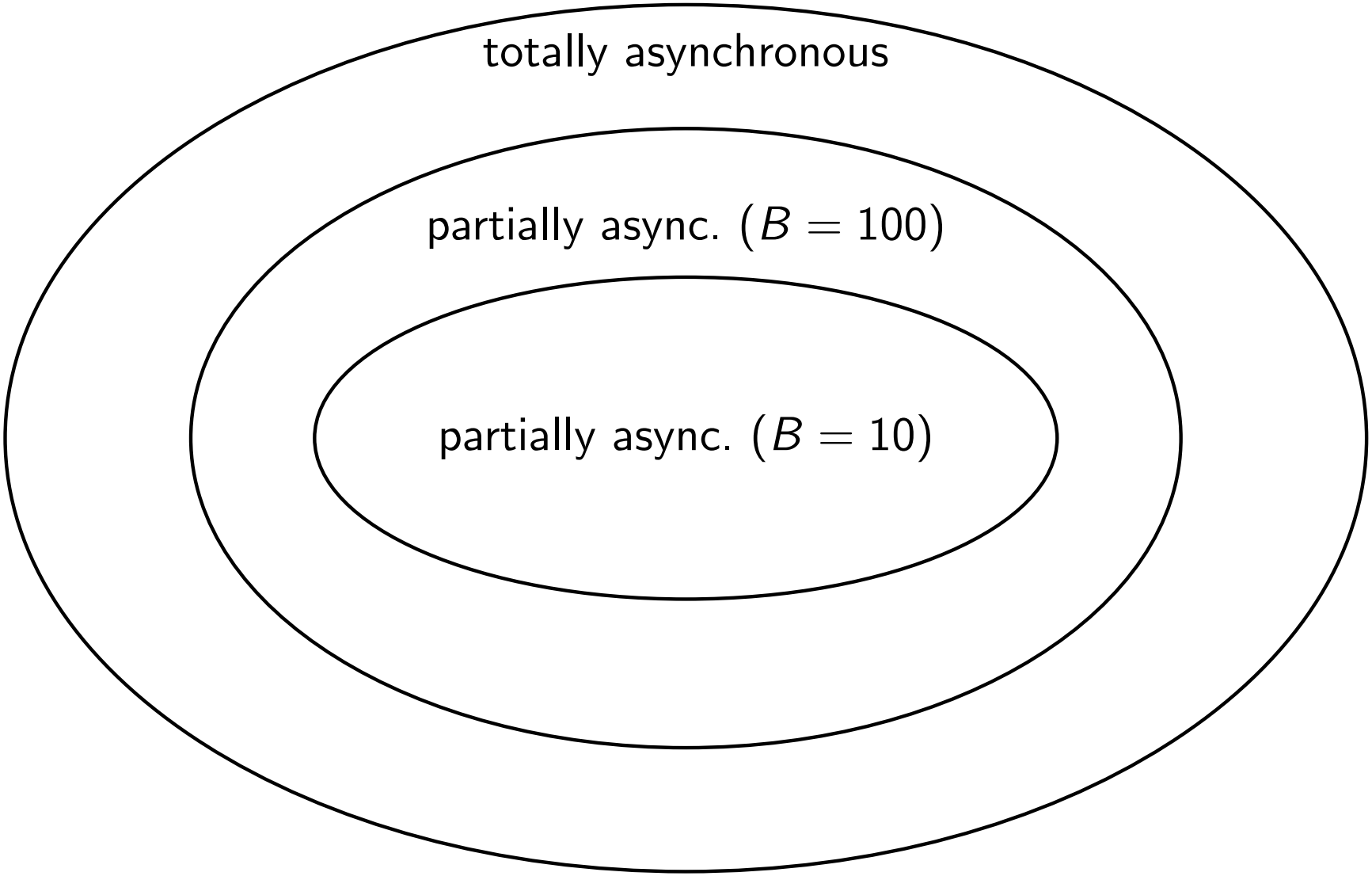
Model for Asynchronous FL

An asynchronous FL algorithm is defined by update events:

$$\mathbf{w}^{(i,t+1)} = \begin{cases} \mathcal{F}^{(i)}\left(\mathbf{w}^{(1,s_{i,1}^{(t)})}, \ldots, \mathbf{w}^{(n,s_{i,n}^{(t)})}\right) & \text{if } t \in \mathcal{U}^{(i)} \\ \mathbf{w}^{(i,t)} & \text{otherwise} \end{cases}$$

- $\mathcal{U}^{(i)}$: set of iterations at which device i is *active*
- $s_{i,i'}^{(t)}$: time stamp of the **last message** received by i from i' at event t
- **delay** $t - s_{i,i'}^{(t)}$: staleness of neighbour information

Totally vs. Partially Asynchronous



Convergence of Asynchronous FL

If the joint operator $\mathcal{F} = \left(\mathcal{F}^{(1)}, \ldots, \mathcal{F}^{(n)}\right)$ is a *pseudo-contraction* in the block-maximum norm,

$$\|\mathcal{F}\,\mathbf{w} - \widehat{\mathbf{w}}\|_\infty \leq \kappa\,\|\mathbf{w} - \widehat{\mathbf{w}}\|_\infty, \quad \kappa \in [0, 1),$$

then the asynchronous algorithm converges. Moreover, for a partially asynchronous algorithm with bound B:

$$\left\|\mathbf{w}^{(t)} - \widehat{\mathbf{w}}\right\|_\infty \leq \kappa^{\frac{t}{2B+1}} \left\|\mathbf{w}^{(0)} - \widehat{\mathbf{w}}\right\|_\infty$$

$\Rightarrow$ **Convergence is faster for smaller κ (better contraction) and smaller B (less staleness).**

Thm. 23–24 in H. R. Feyzmahdavian & M. Johansson, *Asynchronous Iterations in Optimization*, JMLR 2023.

Python Demo – FedGD Sync & Async

Synchronous FedGD

- Server coordinates devices
- Lock-step gradient steps

Asynchronous FedGD

- Devices activate randomly
- Messages carry staleness

Open in Google Colab or your local Python environment.
Try: vary α (edge weight) and observe the effect on the converged local models.

Table of Contents

Feedback from Session 1

Recap and Motivation

Walk-Through: GD for Linear Regression

A Blueprint for FL Algorithms

Synchronous FL Algorithms

Asynchronous FL Algorithms

Wrap-Up

Algorithm Zoo

Algorithm	**Operator** $\mathcal{F}^{(i)}$	**Server?**
FedGD	full gradient step	sync
FedSGD	stochastic gradient step	sync
FedAvg	multi-step SGD + avg	sync
FedProx	proximal step	sync
FedRelax	ridge-regression update	P2P
Async FedGD	gradient + stale messages	none

All algorithms solve (instances of) GTVMin via fixed-point iterations.

Key Takeaways

- GTVMin couples local training via a graph-TV penalty
- FL algorithms are fixed-point iterations on the FL network
- Different choices of update operator $\mathcal{F}^{(i)}$ yield FedGD, FedSGD, FedAvg, FedProx, FedRelax
- Convergence follows from contraction properties of $\mathcal{F}$
- Asynchronous FL is possible when $\mathcal{F}$ is a pseudo-contraction; staleness slows convergence